PRACTICAL 2

CHARUSAT®
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

PRACTICAL 2

Practical 2: Linear and Binary Search: Iterative and Recursive Approaches.

Practical Aim:

To implement linear and binary search using both iterative and recursive approaches.

Problem 1:

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found.

Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

Theory:

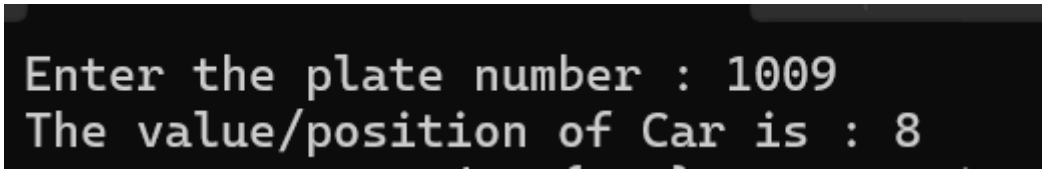
If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?

If the target plate is near the end of a very long list, linear search may need to check almost every plate. To skip some elements, the input would need to have some useful **ordering property**, such as being sorted.

Code-Part 1(Linear search):

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical2

Screenshot :



PRACTICAL 2

Description:

Searching car plate number with the help of linear search in a known plates.

Challenges/difficulties:

There were no major difficulties but where to put the “mid” variable was confusing which comparision argument should be placed first was also confusing. If condition was also little confusing . I learned it and the code successfully runned.

PRACTICAL 2

Part 2(Binary search vehicle number):

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical2

Screenshot :

```
Enter the car number plates and press 0 for exit : 1001
1002
1003
1004
1008
1009
2002
7006
6007
3100
7600
9400
0
Enter the plate number of the car to find: 3100
The car with plate number 3100was at 10th position
```

Description:

Enter plate numbers and searching them using binary search.

Difficulties/challenges:

I didn't faced any challenge in implementation of Linear Search in this practical.

Github Repository Link:

https://github.com/kanan2007s-sys/FDSA_B/tree/main

Conclusion:

Linear search is simple and works on both sorted and unsorted lists. Both iterative and recursive versions can find the first occurrence of the target. However, for a large list, it may require checking many elements, giving a worst-case time complexity of **O(n)**.

PRACTICAL 2

Problem 2

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code.

Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

Theory:

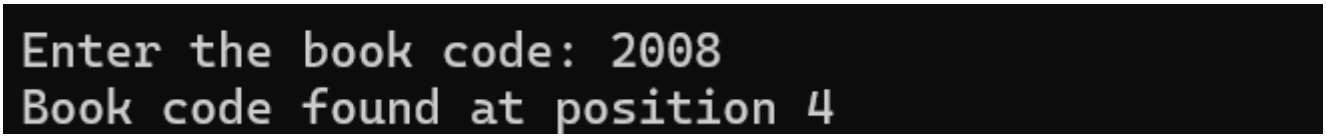
What is the consequence of applying your solution to a list that is not sorted? Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example

Applying binary search to an unsorted list does not necessarily produce a wrong answer every time. It may work accidentally for some inputs, but there is **no guarantee of correctness** because the comparison with the middle element can no longer tell us whether the target lies on the left or right.

Code-Part 1(Binary search):

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical2

Screenshot :



Description:

Searching books in library using its code with the help of binary search .

PRACTICAL 2

Difficulties:

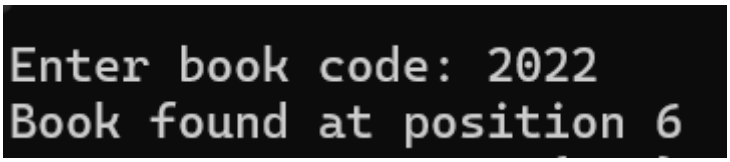
This practical has same concept of Binary search but here I used while loop instead of for loop else the difficulties where less and same as practical 2 part-2.

PRACTICAL 2

Part 2(Binary search with recursion):

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical2

Screenshot :



Description:

Searching books in library from book code, but binary search in recursive function.

Challenges:

I faced challenge in using the binary search logic in function. I could not define what variable to keep as parameter .It was quite difficult to make the code recursive and running .

Repository :

https://github.com/kanan2007s-sys/FDSA_B/tree/main

Conclusion:

Binary search is much faster than linear search for large datasets because it eliminates half of the remaining elements at every step. Both iterative and recursive approaches have a time complexity of O(log n). However, binary search requires the list to be **sorted**; without this condition, its result cannot be trusted.